ورقة العمل (1)

أساسيات لغة البرمجة بايثون (1): المتغيّرات، وأنواع البيانات، والإدخال والإخراج

الدرس: أساسيات لغة البرمجة بايثون **عدد الأسئلة:** 6

النتاجات التي تحقّقها الورقة: (1-6)، (1-5)، (1-4)، (1-3)، (1-2)، (1-1)

اسم الطالب: ..... الشعبة: ..... التاريخ: .....

تذكّر

- تقرأ الدالة `input()` ما يُدخله المستخدم **نصًّا دائمًا**، فإذا أردنا عددًا نحوّله باستخدام `int()`.
- يبدأ المعرّف بحرف أو بشرطة سفلية `_`، ولا يبدأ بعدد، ولا يحتوي على مسافات أو رموز مثل `@` و `-`، ولا يكون كلمة محجوزة.
- تميّز بايثون بين الأحرف الكبيرة والصغيرة: `age` و `Age` متغيّران مختلفان.
- الرمز `#` لكتابة تعليق لا يُنفَّذ، والفاصلة المنقوطة `;` لكتابة أكثر من جملة في سطر، والرمز `\` لكتابة أمر واحد على أكثر من سطر.

السؤال (1-1): حاسبة العمر

اكتب برنامجًا يطلب من المستخدم إدخال **سنة ميلاده**، ثم يحسب عمره بالسنوات ويطبعه.

قبل كتابة البرنامج، اكتب **النموذج الأوّلي** له (المدخلات، والمعالجة، والمخرجات).

النتاج المرتبط

(1-1) أُعرّف النموذج الأوّلي للبرنامج.

(1-6) أتعرّف الصيغة العامة لجملة الإدخال input.

(1-4) أُجري عمليات حسابية على التعابير الحسابية.

✓ الحل

النموذج الأوّلي للبرنامج

المدخلات	المعالجة	المخرجات
سنة الميلاد (birth_year)	العمر = السنة الحالية – سنة الميلاد	طباعة العمر

```
# برنامج لحساب عمر الطالب بالسنوات
CURRENT_YEAR = 2026          # السنة الحالية (قيمة ثابتة)
birth_year = int(input("Enter your birth year: "))
age = CURRENT_YEAR - birth_year
print("Your age is", age, "years")
```

▶ ناتج التنفيذ:

```
Enter your birth year: 2009
Your age is 17 years
```

ملاحظات مهمة:

◄ الدالة `input()` تُرجع ما يكتبه المستخدم نصًّا، لذلك حوّلناه إلى عدد صحيح باستخدام `int()` قبل إجراء عملية الطرح.

◄ كتبنا اسم المتغيّر `CURRENT_YEAR` بأحرف كبيرة للدلالة على أنه **قيمة ثابتة** لا تتغيّر أثناء تنفيذ البرنامج.

◄ السطر الذي يبدأ بالرمز `#` تعليق يوضّح هدف البرنامج، ولا يؤثّر في تنفيذه.

★ **تحدٍّ إضافي:** عدّل البرنامج ليطبع أيضًا: عدد السنوات المتبقية حتى يبلغ المستخدم 18 عامًا، وعمره بالأشهر. ثم احذف `int()` من البرنامج الأصلي، ونفّذه، وفسّر رسالة الخطأ التي تظهر.

(إجابة التحدي في الملحق آخر الملف)

السؤال (2-1): بطاقة تعريف الطالب

اكتب برنامجًا يطبع بطاقة تعريف لطالب تتضمّن: اسمه، ومدرسته، وصفّه، وشعبته، ومادته المفضّلة، مستخدمًا علامات الاقتباس المفردة `'` والمزدوجة `"` والثلاثية `"""`، وطباعة أكثر من مُخرَج في جملة `print` واحدة.

النتاج المرتبط

(2-1) أُبيّن قواعد كتابة الجملة البرمجية بلغة البرمجة بايثون (Python).

(3-1) أُوضّح عناصر لغة البرمجة بايثون (Python)؛ من: ثوابت، ومتغيّرات، ورموز، وتعابير، وعلاقات.

✓ الحل

```
# بطاقة تعريف طالب
print("====== Student Card ======")
print('Name:', "Omar Khaled")
print("School:", 'Swileh Secondary School')
details = """Grade: 11
Section: B
Favorite subject: Digital Skills"""
print(details)
print("==========================")
```

▶ ناتج التنفيذ:

```
====== Student Card ======
Name: Omar Khaled
School: Amman Secondary School
Grade: 11
Section: B
Favorite subject: Digital Skills
==========================
```

ملاحظات مهمة:

◄ عند وضع أكثر من مُخرَج في جملة `print` نفصل بينها بفاصلة، فتُطبع بينها مسافة واحدة.

◄ لا فرق بين علامة الاقتباس المفردة `'` والمزدوجة `"` في كتابة نص من سطر واحد.

◄ تُستخدم علامات الاقتباس الثلاثية `"""` لكتابة نص يمتد على عدة أسطر، وقد خزّنّاه في المتغيّر `details`.

★ **تحدٍّ إضافي:** اطبع الجملة الآتية كما هي تمامًا: `My teacher said 'Practice makes perfect'`. ما نوع علامات الاقتباس التي يجب أن تحيط بالنص من الخارج؟ ولماذا؟

(إجابة التحدي في الملحق آخر الملف)

السؤال (3–1): المعرّفات والكلمات المحجوزة	النتاج المرتبط
صنّف الأسماء الآتية إلى معرّفات **مقبولة** وأخرى **غير مقبولة** في لغة بايثون، مع ذكر السبب: `student_name` ، `2nd_mark` ، `_total` ، `mark-1` ، `while` ، `Avg2` ، `my score` ، `@email` ، `True` ، `average_score` ، `Class`	**(1–2)** أُبيّن قواعد كتابة الجملة البرمجية بلغة البرمجة بايثون (Python). **(1–3)** أُوضّح عناصر لغة البرمجة بايثون (Python)؛ من: ثوابت، ومتغيّرات، ورموز، وتعابير، وعلاقات.

✓ الحل

الاسم	الحكم	السبب
`student_name`	مقبول	يتكوّن من حروف وشرطة سفلية.
`2nd_mark`	غير مقبول	يبدأ بعدد.
`_total`	مقبول	يجوز أن يبدأ المعرّف بالشرطة السفلية.
`mark-1`	غير مقبول	يحتوي على الرمز `-` وهو غير مسموح.

الاسم	الحكم	السبب
`while`	غير مقبول	كلمة محجوزة في بايثون.
`Avg2`	مقبول	يبدأ بحرف، والعدد ليس في بدايته.
`my score`	غير مقبول	يحتوي على مسافة.
`@email`	غير مقبول	يحتوي على الرمز `@`.
`True`	غير مقبول	كلمة محجوزة (قيمة منطقية).
`average_score`	مقبول	اسم واضح يفصل بين كلمتيه بشرطة سفلية.
`Class`	مقبول	بايثون تميّز حالة الأحرف، فـ `Class` تختلف عن الكلمة المحجوزة `class` (ويُفضّل تجنّبه منعًا للالتباس).

مثال يوضّح تمييز حالة الأحرف:

```
age = 16
Age = 17
print(age, Age)
```

▶ ناتج التنفيذ:

```
16 17
```

ملاحظات مهمة:

◄ المتغيّران `age` و `Age` مختلفان؛ لأن بايثون تميّز بين الأحرف الكبيرة والصغيرة.

★ تحدٍّ إضافي: اكتشف الأخطاء في البرنامج الآتي دون تنفيذه، ثم اكتب البرنامج الصحيح:

```
1st_mark = int(input("Enter first mark: "))
class = int(input("Enter second mark: "))
total mark = 1st_mark + class
Print("Total =", total mark)
```

(إجابة التحدي في الملحق آخر الملف)

السؤال (4-1): أنواع البيانات والدالة type()	النتاج المرتبط
	(1-3) أُوضّح عناصر لغة البرمجة بايثون (Python)؛ من: ثوابت،

يمثّل البرنامج الآتي بيانات طالبة. **توقّع** نوع كل متغيّر قبل التنفيذ، ثم نفّذ البرنامج للتحقّق.	ومتغيّرات، ورموز، وتعابير، وعلاقات.

✓ الحل

```
age = 16
name = "Lina"
height = 1.62
is_student = True
mark = "95"
print(type(age))
print(type(name))
print(type(height))
print(type(is_student))
print(type(mark))
```

▶ ناتج التنفيذ:

```
<class 'int'>
<class 'str'>
<class 'float'>
<class 'bool'>
<class 'str'>
```

المتغيّر	النوع	السبب
`age = 16`	`int`	عدد صحيح.
`name = "Lina"`	`str`	نص بين علامتي اقتباس.
`height = 1.62`	`float`	عدد يحتوي على فاصلة عشرية.
`is_student = True`	`bool`	قيمة منطقية (`True` أو `False`).
`mark = "95"`	`str`	بالرغم من أنه رقم، إلا أنه بين علامتي اقتباس فيُعدّ نصًّا.

ملاحظات مهمة:

◄ تحدّد بايثون نوع المتغيّر تلقائيًّا بناءً على القيمة المُسندة إليه، ولا نحتاج إلى التصريح بالنوع.

★ تحدٍّ إضافي: توقّع ناتج كل جملة مما يأتي، ثم تحقّق بالتنفيذ:

```
print(type(10 / 2))
print(type(10 // 2))
print(type(3 > 2))
print("3" + "2")
```

(إجابة التحدي في الملحق آخر الملف)

السؤال (1-5): تحويل درجة الحرارة من سيلسيوس إلى فهرنهايت	النتاج المرتبط
اكتب برنامجًا يطلب إدخال درجة الحرارة في مدينتك اليوم بالسيلسيوس (عددًا صحيحًا)، ثم يحوّلها إلى فهرنهايت وفق العلاقة: `F = C * 9 / 5 + 32`	**(1-6)** أتعرّف الصيغة العامة لجملة الإدخال input. **(1-4)** أُجري عمليات حسابية على التعابير الحسابية. **(1-5)** أكتب كلًّا من العلاقات والعبارات الحسابية والمنطقية باستخدام لغة البرمجة بايثون (Python).

✓ الحل

```
# تحويل درجة الحرارة من سيلسيوس إلى فهرنهايت
celsius = int(input("Enter temperature in Celsius: "))
fahrenheit = celsius * 9 / 5 + 32
print(celsius, "C =", fahrenheit, "F")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 25):

```
Enter temperature in Celsius: 25
25 C = 77.0 F
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 0):

```
Enter temperature in Celsius: 0
0 C = 32.0 F
```

ملاحظات مهمة:

◀ يُنفَّذ الضرب والقسمة أولًا من اليسار إلى اليمين (`25 * 9 = 225` ثم `225 / 5 = 45.0`)، ثم الجمع (`45.0 + 32 = 77.0`).

◀ الناتج عدد عشري (77.0)؛ لأن عامل القسمة `/` يُرجع دائمًا عددًا عشريًّا.

★ تحدٍّ إضافي: اكتب البرنامج العكسي الذي يحوّل من فهرنهايت إلى سيلسيوس وفق العلاقة `C = (F - 32) * 5 / 9`، وجرّبه على درجة غليان الماء 212. لماذا لا يمكن الاستغناء عن الأقواس؟

(إجابة التحدي في الملحق آخر الملف)

السؤال (1-6): حساب معدّل علامات طالب	النتاج المرتبط

اكتب برنامجًا يطلب علامات الطالب في ثلاث مواد (من 100)، ثم يحسب **المجموع** و**المعدّل** ويطبعهما. استخدم الرمز \ لكتابة أمر الجمع على سطرين، والفاصلة المنقوطة ; لكتابة جملتين في سطر واحد.	**(1-2)** أُبيّن قواعد كتابة الجملة البرمجية بلغة البرمجة بايثون (Python). **(1-4)** أُجري عمليات حسابية على التعابير الحسابية. **(1-6)** أتعرّف الصيغة العامة لجملة الإدخال input.

✓ الحل

```
# حساب مجموع ومعدّل علامات ثلاث مواد
arabic_mark = int(input("Arabic mark: "))
math_mark = int(input("Math mark: "))
science_mark = int(input("Science mark: "))
total = arabic_mark + \
        math_mark + science_mark
count = 3; average = total / count
print("Total =", total)
print("Average =", average)
```

▶ ناتج التنفيذ:

```
Arabic mark: 88
Math mark: 92
Science mark: 81
Total = 261
Average = 87.0
```

ملاحظات مهمة:

◄ الرمز \ في نهاية السطر يُخبر مفسّر بايثون أن الأمر يكتمل في السطر التالي.

◄ الفاصلة المنقوطة ; سمحت بكتابة جملتي الإسناد `count = 3` و `average = total / count` في سطر واحد.

★ **تحدٍّ إضافي:** أضف مادة رابعة (الحاسوب) واحسب المعدّل للمواد الأربع. ثم جرّب العلامات 88 و 92 و 81 و 90، وقارن بين ناتج `total / 4` وناتج `total // 4`. ما سبب الاختلاف؟

(إجابة التحدي في الملحق آخر الملف)

ورقة العمل (2)

أساسيات لغة البرمجة بايثون (2): العوامل الحسابية والمتقدمة، والإسناد، والمقارنة، والعوامل المنطقية

الدرس: أساسيات لغة البرمجة بايثون **عدد الأسئلة:** 7

النتاجات التي تحقّقها الورقة: (1-5)، (1-4)، (1-3)

اسم الطالب: ..... الشعبة: ..... التاريخ: .....

تذكّر

- العوامل الحسابية: `+` جمع، `-` طرح، `*` ضرب، `/` قسمة، `%` باقي القسمة، `**` القوّة، `//` القسمة التحتية.
- أولوية التنفيذ: الأقواس، ثم `**` (من اليمين إلى اليسار)، ثم الإشارة السالبة، ثم `* / % //` (من اليسار إلى اليمين)، ثم `+ -`، ثم المقارنة، ثم `not`، ثم `and`، ثم `or`.
- العامل `=` للإسناد (يضع قيمة في متغيّر)، أما `==` فللمقارنة (يسأل: هل القيمتان متساويتان؟).
- ناتج المقارنة والعوامل المنطقية قيمة منطقية: `True` أو `False`.

السؤال (1-2): العوامل الحسابية السبعة

اكتب برنامجًا يطلب من المستخدم إدخال عددين صحيحين، ثم يطبع ناتج تطبيق العوامل الحسابية السبعة عليهما.

النتاج المرتبط

(1-4) أُجري عمليات حسابية على التعابير الحسابية.

(1-5) أكتب كلًّا من العلاقات والعبارات الحسابية والمنطقية باستخدام لغة البرمجة بايثون (Python).

✓ الحل

```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
print("a + b  =", a + b)
print("a - b  =", a - b)
print("a * b  =", a * b)
print("a / b  =", a / b)
print("a % b  =", a % b)
print("a ** b =", a ** b)
print("a // b =", a // b)
```

▶ ناتج التنفيذ:

```
Enter first number: 17
Enter second number: 5
a + b  = 22
a - b  = 12
a * b  = 85
a / b  = 3.4
a % b  = 2
a ** b = 1419857
a // b = 3
```

العامل	الاسم	17 مع 5	التوضيح
+	الجمع	22	17 + 5
-	الطرح	12	17 − 5
*	الضرب	85	17 × 5
/	القسمة	3.4	ناتج القسمة الكامل (عدد عشري)
%	باقي القسمة	2	17 = 5 × 3 + 2
**	القوّة	1419857	17 مرفوعة للقوّة 5
//	القسمة التحتية	3	أقرب عدد صحيح أقل من 3.4 أو يساويه

★ **تحدٍّ إضافي:** توقّع ناتج البرنامج عند إدخال العددين (−17) و (5)، ثم تحقّق بالتنفيذ. فسّر ناتج العاملين `//` و `%`.

(إجابة التحدي في الملحق آخر الملف)

السؤال (2–2): تحويل دقائق الدراسة إلى ساعات ودقائق

سجّل طالب الوقت الذي أمضاه في الدراسة خلال الأسبوع بالدقائق. اكتب برنامجًا يطلب عدد الدقائق ويحوّلها إلى **ساعات ودقائق**.

النتاج المرتبط

(1–4) أُجري عمليات حسابية على التعابير الحسابية.

(1–5) أكتب كلًّا من العلاقات والعبارات الحسابية والمنطقية باستخدام لغة البرمجة بايثون (Python).

✓ الحل

```
total_minutes = int(input("Total study minutes: "))
hours = total_minutes // 60      # عدد الساعات الكاملة
minutes = total_minutes % 60     # الدقائق المتبقية
print(total_minutes, "minutes =", hours, "hours and", minutes, "minutes")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 135):

```
Total study minutes: 135
135 minutes = 2 hours and 15 minutes
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 600):

```
Total study minutes: 600
600 minutes = 10 hours and 0 minutes
```

ملاحظات مهمة:

◄ `135 // 60 = 2`: عدد الساعات الكاملة.

◄ `135 % 60 = 15`: الدقائق المتبقية بعد استبعاد الساعات الكاملة.

★ **تحدٍّ إضافي:** مدّة فيديو تعليمي 3725 ثانية. اكتب برنامجًا يحوّلها إلى ساعات ودقائق وثوانٍ.

(إجابة التحدي في الملحق آخر الملف)

السؤال (3–2): القوّة والجذر: قطر شاشة الهاتف ومدّخرات الطالب

أ) شاشة هاتف عرضها 6 سم وطولها 8 سم. احسب طول قطرها وفق نظرية فيثاغورس:

القطر = (العرض² + الطول²) مرفوعًا للقوّة 0.5.

ب) ادّخر طالب 100 دينار في حساب توفير بنسبة ربح سنوية 5%. احسب رصيده بعد 3 سنوات وفق العلاقة: الرصيد = المبلغ × (100 + النسبة)^السنوات ÷ 100^السنوات.

النتاج المرتبط

(1–4) أُجري عمليات حسابية على التعابير الحسابية.

(1–5) أكتب كلًّا من العلاقات والعبارات الحسابية والمنطقية باستخدام لغة البرمجة بايثون (Python).

✓ الحل

```
# أ) قطر شاشة الهاتف
width = 6
height = 8
diagonal = (width ** 2 + height ** 2) ** 0.5
print("Screen diagonal =", diagonal, "cm")

# ب) مدّخرات الطالب
money = 100
rate = 5
years = 3
balance = money * (100 + rate) ** years / 100 ** years
print("Balance after", years, "years =", balance, "JD")
```

▶ ناتج التنفيذ:

```
Screen diagonal = 10.0 cm
Balance after 3 years = 115.7625 JD
```

ملاحظات مهمة:

◄ رفع العدد للقوّة 0.5 يعني حساب **الجذر التربيعي** له: `100 ** 0.5 = 10.0`.

◄ في العلاقة الثانية تُنفَّذ القوّة `**` أولًا، ثم الضرب والقسمة من اليسار إلى اليمين: `105 ** 3 = 1157625`، ثم `100 * 1157625 = 115762500`، ثم القسمة على `100 ** 3 = 1000000`.

★ **تحدٍّ إضافي:** اكتب برنامجًا يطلب نصف قطر دائرة (عددًا صحيحًا)، ويحسب مساحتها باستخدام الثابت `PI = 3.14` والعلاقة: المساحة = PI × نق². جرّبه على نصف القطر 10.

(إجابة التحدي في الملحق آخر الملف)

السؤال (4–2): عوامل الإسناد: تتبّع مصروف الطالب

مع طالب 10 دنانير. **تتبّع** قيمة المتغيّر `money` بعد كل سطر دون تشغيل البرنامج، ثم نفّذه للتحقّق.

النتاج المرتبط

(1–3) أوضّح عناصر لغة البرمجة بايثون (Python)؛ من: ثوابت، ومتغيّرات، ورموز، وتعابير، وعلاقات.

(1–4) أُجري عمليات حسابية على التعابير الحسابية.

✓ الحل

```
money = 10          # المصروف الأساسي
money += 5          # هدية من الجد
print(money)
money -= 3          # شراء شطيرة وعصير
print(money)
money *= 2          # الأب ضاعف المبلغ
print(money)
money /= 4          # تقسيم المبلغ على 4 أسابيع
print(money)
money //= 2
print(money)
money **= 2
print(money)
money %= 5
print(money)
```

► ناتج التنفيذ:

```
15
12
24
6.0
3.0
9.0
4.0
```

الجملة	تكافئ	قيمة money
`money = 10`	`money = 10`	10
`money += 5`	`money = money + 5`	15
`money -= 3`	`money = money - 3`	12
`money *= 2`	`money = money * 2`	24
`money /= 4`	`money = money / 4`	6.0
`money //= 2`	`money = money // 2`	3.0
`money **= 2`	`money = money ** 2`	9.0
`money %= 5`	`money = money % 5`	4.0

ملاحظات مهمة:

◄ بعد عملية القسمة `/=` أصبحت القيمة عددًا عشريًا (6.0)، وبقيت كذلك في العمليات التالية.

★ **تحدٍّ إضافي:** ما القيمة النهائية للمتغيّر x؟ تتبّع دون تشغيل، ثم تحقّق:

```
x = 7
x += 3
x *= 2
x //= 3
x %= 4
x **= 3
print(x)
```

(إجابة التحدي في الملحق آخر الملف)

السؤال (5-2): أولوية العوامل وترابطها

احسب ناتج كل تعبير مما يأتي **يدويًا خطوة بخطوة** وفق أولوية العوامل، ثم تحقّق بالتنفيذ:

```
7 + 3 * 2 ** 2
20 / 4 * 2
2 ** 3 ** 2
(10 - 4) * 3 % 4
-3 ** 2
17 // 5 * 5 + 17 % 5
```

النتاج المرتبط

(4-1) أُجري عمليات حسابية على التعابير الحسابية.

✓ **الحل**

التعبير	خطوات الحل	الناتج
`7 + 3 * 2 ** 2`	القوّة أولًا: `2 ** 2 = 4`، ثم الضرب: `3 * 4 = 12`، ثم الجمع: `7 + 12`	19

التعبير	خطوات الحل	الناتج
`20 / 4 * 2`	للعاملين الأولوية نفسها فننفّذ من اليسار إلى اليمين: `20 / 4 = 5.0`، ثم `5.0 * 2`	`10.0`
`2 ** 3 ** 2`	القوّة تُنفَّذ من اليمين إلى اليسار: `3 ** 2 = 9`، ثم `2 ** 9`	`512`
`(10 - 4) * 3 % 4`	الأقواس: `6`، ثم من اليسار إلى اليمين: `6 * 3 = 18`، ثم `18 % 4`	`2`
`-3 ** 2`	القوّة أعلى أولوية من الإشارة السالبة: `3 ** 2 = 9`، ثم تُطبَّق الإشارة	`-9`
`17 // 5 * 5 + 17 % 5`	`17 // 5 = 3`، ثم `3 * 5 = 15`، ثم `17 % 5 = 2`، ثم `15 + 2`	`17`

التحقّق بالتنفيذ:

```
print(7 + 3 * 2 ** 2)
print(20 / 4 * 2)
print(2 ** 3 ** 2)
print((10 - 4) * 3 % 4)
print(-3 ** 2)
print(17 // 5 * 5 + 17 % 5)
```

▶ ناتج التنفيذ:

```
19
10.0
512
2
-9
17
```

ملاحظات مهمة:

◄ لاحظ في التعبير الأخير أن ناتج `a // b * b + a % b` يساوي دائمًا العدد الأصلي `a`، وهذا يوضّح العلاقة بين القسمة التحتية وباقي القسمة.

★ **تحدٍّ إضافي:** ما ناتج التعبير `2 + 3 * 4 ** 2`؟ أضف أقواسًا إلى التعبير نفسه ليصبح ناتجه 80، ثم أضف أقواسًا ليصبح ناتجه 400.

(إجابة التحدي في الملحق آخر الملف)

السؤال (6-2): عوامل المقارنة والفرق بين = و ==

حصل أحمد على 85، وحصلت سلمى على 92 في اختبار المهارات الرقمية. اكتب برنامجًا يقارن بين العلامتين باستخدام عوامل المقارنة الستة.

النتاج المرتبط

(1-5) أكتب كلًّا من العلاقات والعبارات الحسابية والمنطقية

	باستخدام لغة البرمجة بايثون (Python). **(1-3)** أُوضّح عناصر لغة البرمجة بايثون (Python)؛ من: ثوابت، ومتغيّرات، ورموز، وتعابير، وعلاقات.

✓ الحل

```
ahmad = 85        # = عامل إسناد: يضع القيمة في المتغيّر
salma = 92
print("ahmad == salma :", ahmad == salma)   # == عامل مقارنة
print("ahmad != salma :", ahmad != salma)
print("ahmad > salma  :", ahmad > salma)
print("ahmad < salma  :", ahmad < salma)
print("ahmad >= 85    :", ahmad >= 85)
print("salma <= 90    :", salma <= 90)
```

▶ ناتج التنفيذ:

```
ahmad == salma : False
ahmad != salma : True
ahmad > salma  : False
ahmad < salma  : True
ahmad >= 85    : True
salma <= 90    : False
```

ملاحظات مهمة:

◄ `ahmad = 85` جملة **إسناد** تضع القيمة 85 في المتغيّر، أما `ahmad == salma` فتعبير **مقارنة** ناتجه True أو False.

◄ `ahmad >= 85` ناتجها True لأن القيمتين متساويتان، والعامل `>=` يعني أكبر من **أو يساوي**.

★ تحدٍّ إضافي: توقّع ناتج كل جملة مما يأتي، ثم تحقّق بالتنفيذ:

```
print(5 == 5.0)
print("5" == 5)
print(10 / 2 == 5)
print(7 // 2 != 3)
```

(إجابة التحدي في الملحق آخر الملف)

السؤال (7-2): العوامل المنطقية وأولويتها	النتاج المرتبط
أ) أكمل جدول الصواب والخطأ للعوامل المنطقية. ب) تتبّع ناتج البرنامج الآتي دون تشغيله، ثم تحقّق بالتنفيذ.	**(1-5)** أكتب كلًّا من العلاقات والعبارات الحسابية والمنطقية

باستخدام لغة البرمجة بايثون (Python). **(1-4)** أُجري عمليات حسابية على التعابير الحسابية.	

✓ الحل

أ) جدول الصواب والخطأ

A	B	A and B	A or B	not A
True	True	True	True	False
True	False	False	True	False
False	True	False	True	True
False	False	False	False	True

ب) البرنامج وناتج تنفيذه:

```
age = 16
has_id = True
mark = 74
print(age >= 16 and has_id)
print(mark >= 90 or mark >= 70)
print(not (mark < 50))
print(10 % 3 == 1 and 2 ** 3 > 7)
print(15 // 4 == 4 or 5 > 9)
print(3 < 5 or 8 < 2 and not True)
```

▶ ناتج التنفيذ:

```
True
True
True
True
False
True
```

التعبير	التحليل	الناتج
age >= 16 and has_id	True and True	True
mark >= 90 or mark >= 70	False or True	True
not (mark < 50)	not False	True
10 % 3 == 1 and 2 ** 3 > 7	(1 == 1) and (8 > 7) → True and True	True
15 // 4 == 4 or 5 > 9	(3 == 4) or (5 > 9) → False or False	False
3 < 5 or 8 < 2 and not True	تُنفَّذ not ثم and ثم or: True or (False and False) ثم True or False	True

ملاحظات مهمة:

◄ أولوية العوامل المنطقية: `not` أولًا، ثم `and`، ثم `or`. وعوامل الحساب والمقارنة تُنفَّذ قبل العوامل المنطقية.

★ **تحدٍّ إضافي:** إذا كانت `x = 4` و `y = 1`، فما ناتج كل من الجملتين الآتيتين؟ ولماذا اختلف الناتج؟

```
print(not x > 3 and y == 0)
print(not (x > 3 and y == 0))
```

(إجابة التحدي في الملحق آخر الملف)

ورقة العمل (3)

الجمل الشرطية: if و else و elif، والمعاملات المنطقية، والجمل المتداخلة

الدرس: الجمل الشرطية **عدد الأسئلة:** 10

النتاجات التي تحقّقها الورقة: (2-2)، (2-1)، (1-5)، (1-4)، (1-2)

اسم الطالب: ..... الشعبة: ..... التاريخ: .....

تذكّر

- توضع النقطتان `:` بعد الشرط وبعد `else`، ونسيانهما يسبّب خطأً من نوع SyntaxError.
- الأوامر التابعة للشرط تُكتب بمسافة بادئة (indentation) متساوية، وهي التي تحدّد الكتلة البرمجية.
- في `if ... elif ... else` يُنفَّذ أول شرط صحيح فقط، ثم يُتجاهل الباقي.
- `and`: يجب تحقّق الشرطين معًا. `or`: يكفي تحقّق أحدهما. `not`: تنفيذ الأمر عند عدم تحقّق الشرط.

السؤال (1-3): الجملة الشرطية if: هل نجح الطالب؟

اكتب برنامجًا يطلب علامة الطالب، ويطبع رسالة تهنئة إذا كانت العلامة 50 فأكثر، ثم يطبع عبارة "Thank you" في جميع الحالات.

النتاج المرتبط

(2-2) أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.

(2-1) أُبيّن قواعد كتابة الجملة البرمجية بلغة البرمجة بايثون (Python).

✓ الحل

```
mark = int(input("Enter your mark: "))
if mark >= 50:
    print("Congratulations, you passed!")
print("Thank you")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 75):

```
Enter your mark: 75
Congratulations, you passed!
Thank you
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 40):

```
Enter your mark: 40
Thank you
```

ملاحظات مهمة:

◄ جملة `print("Congratulations...")` تابعة للشرط لأنها مكتوبة بمسافة بادئة، فلا تُنفَّذ إلا إذا تحقّق الشرط.

◄ جملة `print("Thank you")` ليست داخل الكتلة (بلا مسافة بادئة)، لذلك تُنفَّذ دائمًا.

★ **تحدٍّ إضافي:** ماذا يحدث لو أضفنا مسافة بادئة قبل جملة `print("Thank you")`؟ نفّذ البرنامج المعدّل على العلامة 40 ثم على العلامة 75.

(إجابة التحدي في الملحق آخر الملف)

السؤال (2–3): الجملة الشرطية if else: توزيع الطلاب في مجموعات ثنائية

يريد المعلّم توزيع طلاب الشعبة في مجموعات ثنائية. اكتب برنامجًا يطلب عدد الطلاب، ويطبع عدد الأزواج، وهل يمكن توزيعهم جميعًا في أزواج (العدد زوجي) أم سيبقى طالب ينضم إلى مجموعة ثلاثية (العدد فردي).

النتاج المرتبط

(2–2) أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.

(1–4) أُجري عمليات حسابية على التعابير الحسابية.

✓ الحل

```
students = int(input("Number of students: "))
pairs = students // 2
if students % 2 == 0:
    print("Even number:", pairs, "pairs")
else:
    print("Odd number:", pairs, "pairs, and 1 student joins a group of 3")
```

► ناتج التنفيذ – التجربة 1 (المدخلات: 28):

```
Number of students: 28
Even number: 14 pairs
```

► ناتج التنفيذ – التجربة 2 (المدخلات: 31):

```
Number of students: 31
Odd number: 15 pairs, and 1 student joins a group of 3
```

ملاحظات مهمة:

◄ العدد زوجي إذا كان باقي قسمته على 2 يساوي صفرًا: `students % 2 == 0`.

★ **تحدٍّ إضافي:** اكتب برنامجًا يطلب سعر سلعة والمبلغ الذي يملكه الطالب، ثم يطبع "You can buy it" والمبلغ المتبقي معه، أو "Not enough money" والمبلغ الناقص.

(إجابة التحدي في الملحق آخر الملف)

السؤال (3-3): الجملة الشرطية if elif: إشارة المرور	النتاج المرتبط
اكتب برنامجًا يطلب لون إشارة المرور (red أو yellow أو green)، ويطبع التصرّف المناسب: Stop أو Get ready أو Go.	(2-2) أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.

✓ الحل

```
color = input("Traffic light color (red/yellow/green): ")
if color == "red":
    print("Stop")
elif color == "yellow":
    print("Get ready")
elif color == "green":
    print("Go")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: yellow):

```
Traffic light color (red/yellow/green): yellow
Get ready
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: blue):

```
Traffic light color (red/yellow/green): blue
```

ملاحظات مهمة:

◄ عند إدخال `blue` لم يُطبع شيء؛ لأن أيًّا من الشروط لم يتحقّق، ولا توجد جملة `else`.

◄ تُقارن النصوص حرفًا بحرف، ولا حاجة إلى `int()` هنا لأن المُدخل نص.

★ **تحدٍّ إضافي:** أضف إلى البرنامج جملة `else` تطبع "Invalid color". ثم فسّر: ماذا يطبع البرنامج إذا أدخل المستخدم `Red` بحرف كبير؟

(إجابة التحدي في الملحق آخر الملف)

السؤال (4-3): الجملة الشرطية if elif else: تقدير الطالب	النتاج المرتبط
اكتب برنامجًا يطلب علامات الطالب في ثلاث مواد، ويحسب معدّله، ثم يطبع تقديره وفق الجدول الآتي:	(2-2) أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها. (1-4) أُجري عمليات حسابية على التعابير الحسابية.

المعدّل	التقدير
90 فأكثر	Excellent

المعدّل	التقدير
80 إلى أقل من 90	Very Good
70 إلى أقل من 80	Good
50 إلى أقل من 70	Pass
أقل من 50	Fail

✓ الحل

```
m1 = int(input("Mark 1: "))
m2 = int(input("Mark 2: "))
m3 = int(input("Mark 3: "))
average = (m1 + m2 + m3) / 3
print("Average =", average)
if average >= 90:
    print("Grade: Excellent")
elif average >= 80:
    print("Grade: Very Good")
elif average >= 70:
    print("Grade: Good")
elif average >= 50:
    print("Grade: Pass")
else:
    print("Grade: Fail")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 81، 92، 88):

```
Mark 1: 88
Mark 2: 92
Mark 3: 81
Average = 87.0
Grade: Very Good
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 39، 60، 45):

```
Mark 1: 45
Mark 2: 60
Mark 3: 39
Average = 48.0
Grade: Fail
```

ملاحظات مهمة:

◄ الأقواس في `(m1 + m2 + m3) / 3` ضرورية؛ لأن القسمة أعلى أولوية من الجمع.

◄ لا نحتاج إلى كتابة `average >= 80 and average < 90`؛ لأن الوصول إلى `elif average >= 80` يعني أن الشرط السابق `average >= 90` لم يتحقّق.

★ **تحدٍّ إضافي:** ماذا يحدث لو رتّبنا الشروط ترتيبًا عكسيًّا وبدأنا بالشرط `average >= 50`؟ جرّب بمعدّل 95.

(إجابة التحدي في الملحق آخر الملف)

السؤال (3-5): المعامل المنطقي and: منحة التفوّق	النتاج المرتبط
تمنح المدرسة منحة للطالب إذا كان معدّله 90 فأكثر، و عدد أيام غيابه لا يزيد على 5 أيام. اكتب برنامجًا يحدّد هل يستحق الطالب المنحة.	(2-1) أكتب جملًا شرطية مُركّبة ومترابطة باستخدام المعاملات المنطقية (and، و or، و not) في لغة البرمجة بايثون (Python). (2-2) أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.

✓ الحل

```
average = int(input("Average: "))
absences = int(input("Days of absence: "))
if average >= 90 and absences <= 5:
    print("You get the scholarship")
else:
    print("Sorry, you are not eligible")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 3 ،94):

```
Average: 94
Days of absence: 3
You get the scholarship
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 8 ،94):

```
Average: 94
Days of absence: 8
Sorry, you are not eligible
```

ملاحظات مهمة:

◄ مع and يجب أن يتحقّق الشرطان معًا. في التجربة الثانية المعدّل مرتفع، لكن الغياب 8 أيام، فلم تتحقّق المنحة.

★ **تحدٍّ إضافي:** يمنح متجر خصمًا بنسبة 10% إذا كانت قيمة الفاتورة 50 دينارًا فأكثر، وكان عدد القطع 3 فأكثر. اكتب برنامجًا يطبع قيمة الخصم والمبلغ النهائي، أو "No discount".

(إجابة التحدي في الملحق آخر الملف)

السؤال (3-6): المعامل المنطقي or: تذكرة دخول المتحف	النتاج المرتبط
	(2-1) أكتب جملًا شرطية مُركّبة ومترابطة باستخدام المعاملات

يقدّم متحف دخولًا مجانيًّا للأطفال دون 6 سنوات **أو** لكبار السن (60 سنة فأكثر)، أما بقية الزوّار فيدفعون 2 دينار. اكتب برنامجًا يطلب عمر الزائر ويطبع سعر تذكرته.	المنطقية (and، و or، و not) في لغة البرمجة بايثون (Python). **(2-2)** أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.

✓ الحل

```
age = int(input("Visitor age: "))
if age < 6 or age >= 60:
    print("Free ticket")
else:
    print("Ticket price: 2 JD")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 4):

```
Visitor age: 4
Free ticket
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 65):

```
Visitor age: 65
Free ticket
```

▶ ناتج التنفيذ – التجربة 3 (المدخلات: 30):

```
Visitor age: 30
Ticket price: 2 JD
```

ملاحظات مهمة:

◄ مع `or` يكفي أن يتحقّق أحد الشرطين. العمر 30 لا يحقّق أيًّا منهما، فيُنفَّذ `else`.

★ **تحدٍّ إضافي:** اكتب برنامجًا يطلب اسم اليوم بالإنجليزية، ويطبع "Weekend" إذا كان اليوم Friday أو Saturday، وإلا يطبع "School day".

(إجابة التحدي في الملحق آخر الملف)

السؤال (7-3): المعامل المنطقي not: إعادة كتاب المكتبة	**النتاج المرتبط**
اكتب برنامجًا يسأل الطالب هل أعاد كتاب المكتبة (yes/no)، ويخزّن الإجابة في متغيّر منطقي، ثم يستخدم `not` لطباعة تنبيه إذا لم يُعِد الكتاب.	**(2-1)** أكتب جملًا شرطية مُركّبة ومترابطة باستخدام المعاملات المنطقية (and، و or، و not) في لغة البرمجة بايثون (Python). **(1-5)** أكتب كلًّا من العلاقات والعبارات الحسابية والمنطقية

	باستخدام لغة البرمجة بايثون (Python).

✓ الحل

```
answer = input("Did you return the library book? (yes/no): ")
returned = answer == "yes"       # متغيّر منطقي: True أو False
if not returned:
    print("Please return the book to avoid a fine")
else:
    print("Thank you for returning the book")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: no):

```
Did you return the library book? (yes/no): no
Please return the book to avoid a fine
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: yes):

```
Did you return the library book? (yes/no): yes
Thank you for returning the book
```

ملاحظات مهمة:

◄ في الجملة `returned = answer == "yes"` تُنفَّذ المقارنة `==` أولًا، ثم يُسند ناتجها (True أو False) إلى المتغيّر `returned`.

◄ `not returned` تكون True عندما تكون قيمة `returned` هي False.

★ **تحدٍّ إضافي:** أعد كتابة الشرط `if not returned:` بطريقتين أخريين دون استخدام `not`.

(إجابة التحدي في الملحق آخر الملف)

السؤال (8–3): الجمل الشرطية المتداخلة: الدخول إلى منصّة المدرسة	النتاج المرتبط
اكتب برنامجًا يطلب اسم المستخدم وكلمة المرور للدخول إلى منصّة المدرسة. يتحقّق أولًا من اسم المستخدم (student11)، فإذا كان صحيحًا يتحقّق من كلمة المرور (py2026)، ويطبع رسالة مناسبة لكل حالة.	**(2–1)** أكتب جملًا شرطية مُركّبة ومترابطة باستخدام المعاملات المنطقية (and، و or، و not) في لغة البرمجة بايثون (Python). **(2–2)** أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.

✓ الحل

```
username = input("Username: ")
password = input("Password: ")
if username == "student11":
    if password == "py2026":
        print("Welcome to the school platform")
    else:
        print("Wrong password")
else:
    print("Username not found")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: student11، py2026):

```
Username: student11
Password: py2026
Welcome to the school platform
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: student11، 1234):

```
Username: student11
Password: 1234
Wrong password
```

▶ ناتج التنفيذ – التجربة 3 (المدخلات: ali، py2026):

```
Username: ali
Password: py2026
Username not found
```

ملاحظات مهمة:

◄ جملة `if` الداخلية لا تُفحص إلا إذا تحقّق شرط `if` الخارجية. لاحظ أن الكتلة الداخلية لها مستويان من المسافة البادئة.

★ تحدٍّ إضافي: أعد كتابة البرنامج باستخدام شرط واحد مع المعامل `and`. ما الذي نخسره مقارنةً بالجمل المتداخلة؟

(إجابة التحدي في الملحق آخر الملف)

السؤال (9–3): سنة كبيسة: دمج and و or مع باقي القسمة	النتاج المرتبط
تكون السنة كبيسة (366 يومًا) إذا قبلت القسمة على 4 ولم تقبل القسمة على 100، أو إذا قبلت القسمة على 400. اكتب برنامجًا يطلب سنة ويحدّد هل هي كبيسة.	(2–1) أكتب جملًا شرطية مُركّبة ومترابطة باستخدام المعاملات المنطقية (and، و or، و not) في لغة البرمجة بايثون (Python). (1–4) أُجري عمليات حسابية على التعابير الحسابية. (1–5) أكتب كلًّا من العلاقات والعبارات الحسابية والمنطقية

	باستخدام لغة البرمجة بايثون (Python).

✓ الحل

```
year = int(input("Enter a year: "))
if (year % 4 == 0 and year % 100 != 0) or year % 400 == 0:
    print(year, "is a leap year (366 days)")
else:
    print(year, "is not a leap year (365 days)")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 2028):

```
Enter a year: 2028
2028 is a leap year (366 days)
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 1900):

```
Enter a year: 1900
1900 is not a leap year (365 days)
```

▶ ناتج التنفيذ – التجربة 3 (المدخلات: 2000):

```
Enter a year: 2000
2000 is a leap year (366 days)
```

السنة	÷ 4	÷ 100	÷ 400	النتيجة
2028	نعم	لا	لا	كبيسة (الجزء الأول من الشرط صحيح)
1900	نعم	نعم	لا	ليست كبيسة
2000	نعم	نعم	نعم	كبيسة (الجزء الثاني من الشرط صحيح)

★ تحدٍّ إضافي: اكتب البرنامج نفسه باستخدام جمل شرطية متداخلة بدلًا من `and` و `or`.

(إجابة التحدي في الملحق آخر الملف)

السؤال (10–3): اكتشاف الأخطاء وجملة pass	النتاج المرتبط
أ) اكتشف الأخطاء في البرنامج الآتي دون تنفيذه، ثم اكتب البرنامج الصحيح:	(2–2) أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.

(1-2) أُبيّن قواعد كتابة الجملة البرمجية بلغة البرمجة بايثون (Python).

```
mark = input("Enter mark: ")
if mark = 100
print("Full mark")
Else:
    print("Keep trying")
```

ب) ما فائدة جملة `pass` في البرنامج الآتي؟ وما ناتج تنفيذه؟

```
mark = 95
if mark >= 90:
    pass        # سنضيف رسالة التهنئة لاحقًا
print("Program finished")
```

✓ الحل

السطر	الخطأ	التصحيح
1	المُدخل نص، ولا يمكن مقارنته بالعدد 100	`int(input("Enter mark: "))`
2	استخدام `=` (إسناد) بدلًا من `==` (مقارنة)	`mark == 100`
2	نسيان النقطتين بعد الشرط	`if mark == 100:`
3	لا توجد مسافة بادئة قبل الجملة التابعة للشرط	إضافة أربع مسافات قبل `print`
4	`Else` بحرف كبير	`else:`

أ) البرنامج الصحيح:

```
mark = int(input("Enter mark: "))
if mark == 100:
    print("Full mark")
else:
    print("Keep trying")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 100):

```
Enter mark: 100
Full mark
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 87):

```
Enter mark: 87
Keep trying
```

ب) ناتج البرنامج الذي يحتوي على pass:

```
mark = 95
if mark >= 90:
    pass        # سنضيف رسالة التهنئة لاحقًا
print("Program finished")
```

▶ ناتج التنفيذ:

```
Program finished
```

ملاحظات مهمة:

◄ `pass` جملة لا تفعل شيئًا، نضعها مكان كتلة لم نكتبها بعد، حتى لا يظهر خطأ؛ لأن بايثون لا تقبل كتلة فارغة بعد النقطتين.

★ **تحدٍّ إضافي:** ماذا يحدث لو حذفنا جملة `pass` من البرنامج (ب) وتركنا جملة `if` بلا كتلة؟

(إجابة التحدي في الملحق آخر الملف)

ورقة العمل (4)

الحلقات: while و for و range()، وجمل التحكم break و continue و else و pass

الدرس: الحلقات (Loops) **عدد الأسئلة:** 10

النتاجات التي تحقّقها الورقة: (3-5)، (3-4)، (3-3)، (3-2)، (3-1)، (2-2)، (1-4)، (1-1)

اسم الطالب: ..... الشعبة: ..... التاريخ: .....

تذكّر

- حلقة `while` تكرّر الأوامر **ما دام الشرط صحيحًا**، ويجب تغيير قيمة العدّاد داخلها حتى لا تصبح حلقة لانهائية.
- حلقة `for` تمرّ على عناصر متتابعة (sequence) مثل أعداد `range()` أو أحرف نص.
- `range(a)` من 0 إلى a−1، و `range(a, b)` من a إلى b−1، و `range(a, b, c)` من a إلى b−1 بخطوة c.
- `break` تُوقف الحلقة كاملة، و `continue` تتخطّى الدورة الحالية فقط، و `else` تُنفَّذ عند انتهاء الحلقة دون `break`.

السؤال (1-4): حلقة while: العدّ التنازلي لإطلاق صاروخ

في مشروع العلوم، اكتب برنامجًا يعدّ تنازليًّا من 10 إلى 1 باستخدام حلقة `while`، ثم يطبع "Launch!".

النتاج المرتبط

(3-1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python).

(3-4) أتعرّف جمل التكرار وأذكر أنواعها.

✓ الحل

```
count = 10
while count > 0:
    print(count)
    count -= 1          # إنقاص العدّاد في كل دورة
print("Launch!")
```

▶ ناتج التنفيذ:

```
10
9
8
7
6
5
4
3
2
1
Launch!
```

ملاحظات مهمة:

◄ مكوّنات حلقة while: **قيمة ابتدائية** للعدّاد (`count = 10`)، و**شرط** الاستمرار (`count > 0`)، و**تحديث** العدّاد (`count -= 1`).

◄ عندما تصبح قيمة `count` صفرًا يصبح الشرط False، فتخرج الحلقة ويُنفَّذ ما بعدها.

★ **تحدٍّ إضافي:** عدّل البرنامج ليعدّ تنازليًّا بالأعداد الزوجية فقط من 10 إلى 2. وماذا يحدث لو نسينا السطر `count -= 1`؟

(إجابة التحدي في الملحق آخر الملف)

السؤال (2–4): حلقة while مع قيمة توقّف: معدّل الاختبارات القصيرة

اكتب برنامجًا يطلب من الطالب إدخال علاماته في الاختبارات القصيرة (من 10) واحدة تلو الأخرى، ويتوقّف عند إدخال **–1**، ثم يطبع عدد الاختبارات ومجموع العلامات والمعدّل.

النتاج المرتبط

(3–1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python).

(3–2) أستخدم أكثر الهياكل البرمجية (مثل: الحلقات، والجمل الشرطية) مناسبةً لحلّ مشكلات مُعيّنة بكفاءة.

✓ الحل

```
total = 0
count = 0
mark = int(input("Enter mark (-1 to stop): "))
while mark != -1:
    total += mark
    count += 1
    mark = int(input("Enter mark (-1 to stop): "))
print("Number of quizzes:", count)
print("Total:", total)
if count > 0:
    print("Average:", total / count)
```

▶ ناتج التنفيذ:

```
Enter mark (-1 to stop): 8
Enter mark (-1 to stop): 9
Enter mark (-1 to stop): 7
Enter mark (-1 to stop): 10
Enter mark (-1 to stop): -1
Number of quizzes: 4
Total: 34
Average: 8.5
```

ملاحظات مهمة:

◄ استخدمنا `while` لأننا **لا نعرف مسبقًا** عدد العلامات التي سيُدخلها الطالب.

◄ الشرط `if count > 0` يمنع القسمة على صفر إذا أدخل الطالب −1 مباشرة.

★ تحدٍّ إضافي: عدّل البرنامج ليطبع أيضًا **أعلى** علامة أدخلها الطالب.

(إجابة التحدي في الملحق آخر الملف)

السؤال (3–4): حلقة for والدالة range() بصيغها الثلاث

أ) اطبع العبارة "I love Python" ثلاث مرات باستخدام `range(n)`.

ب) اطبع جدول الضرب لعدد يُدخله المستخدم من 1 إلى 10 باستخدام `range(a, b)`.

ج) اطبع الأعداد الفردية الأقل من 10 باستخدام `range(a, b, c)`.

النتاج المرتبط

(3–1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python).

(3–4) أتعرّف جمل التكرار وأذكر أنواعها.

✓ الحل

```
# أ) range(n): من 0 إلى n-1
for i in range(3):
    print("I love Python")

# ب) range(a, b): من a إلى b-1
num = int(input("Multiplication table of: "))
for i in range(1, 11):
    print(num, "x", i, "=", num * i)

# ج) range(a, b, c): من a إلى b-1 بخطوة c
for n in range(1, 10, 2):
    print(n)
```

▶ ناتج التنفيذ:

```
I love Python
I love Python
I love Python
Multiplication table of: 7
7 x 1 = 7
7 x 2 = 14
7 x 3 = 21
7 x 4 = 28
7 x 5 = 35
7 x 6 = 42
7 x 7 = 49
7 x 8 = 56
7 x 9 = 63
7 x 10 = 70
1
3
5
7
9
```

الصيغة	مثال	الأعداد الناتجة
`range(a)`	`range(5)`	`0, 1, 2, 3, 4`
`range(a, b)`	`range(1, 5)`	`1, 2, 3, 4`
`range(a, b, c)`	`range(1, 5, 2)`	`1, 3`

ملاحظات مهمة:

◄ القيمة النهائية b **لا تدخل** في السلسلة، لذلك كتبنا `range(1, 11)` للوصول إلى 10.

★ **تحدٍّ إضافي:** ما الأعداد التي تولّدها كل دالة مما يأتي؟ تحقّق باستخدام حلقة for: `range(4)` و `range(2, 8)` و `range(0, 20, 5)` و `range(10, 0, -2)`.

(إجابة التحدي في الملحق آخر الملف)

السؤال (4–4): التراكم في حلقة for: مجموع الأعداد ومضروب العدد

اكتب برنامجًا يطلب عددًا صحيحًا موجبًا n، ثم يحسب **مجموع** الأعداد من 1 إلى n، و**حاصل ضربها** (مضروب العدد n).

النتاج المرتبط

(3–1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python).

(4–1) أُجري عمليات حسابية على التعابير الحسابية.

✓ الحل

```
n = int(input("Enter a positive number: "))
total = 0
factorial = 1
for i in range(1, n + 1):
    total += i
    factorial *= i
print("Sum from 1 to", n, "=", total)
print("Factorial of", n, "=", factorial)
```

▶ ناتج التنفيذ:

```
Enter a positive number: 5
Sum from 1 to 5 = 15
Factorial of 5 = 120
```

factorial	total	i
1 × 1 = 1	0 + 1 = 1	1
1 × 2 = 2	1 + 2 = 3	2
2 × 3 = 6	3 + 3 = 6	3
6 × 4 = 24	6 + 4 = 10	4
24 × 5 = 120	10 + 5 = 15	5

ملاحظات مهمة:

◄ يبدأ متغيّر المجموع بالقيمة 0، ويبدأ متغيّر حاصل الضرب بالقيمة **1**؛ لأن البدء بالصفر يجعل الناتج صفرًا دائمًا.

★ **تحدٍّ إضافي:** احسب 2 مرفوعة للقوّة 10 باستخدام حلقة for (ضرب متكرّر)، ثم قارن الناتج بناتج العامل **.

(إجابة التحدي في الملحق آخر الملف)

السؤال (5–4): حلقة for على أحرف نص: عدّ حرف في كلمة	النتاج المرتبط
النص متتابعة من الأحرف. اكتب برنامجًا يطلب كلمة بالإنجليزية، ويعدّ كم مرة ظهر فيها الحرف a.	(3–1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python). (3–2) أستخدم أكثر الهياكل البرمجية (مثل: الحلقات، والجمل الشرطية) مناسبةً لحلّ مشكلات مُعيّنة بكفاءة.

✓ الحل

```
word = input("Enter a word: ")
count = 0
for letter in word:
    if letter == "a":
        count += 1
print("Letter a appears", count, "times")
```

▶ ناتج التنفيذ:

```
Enter a word: banana
Letter a appears 3 times
```

ملاحظات مهمة:

◄ في كل دورة يأخذ المتغيّر `letter` حرفًا من الكلمة بالترتيب: b ثم a ثم n ...، وتفحص جملة `if` الحرف.

★ تحدٍّ إضافي: عدّل البرنامج ليعدّ أحرف العلّة (a, e, i, o, u) في الكلمة باستخدام المعامل `or`.

(إجابة التحدي في الملحق آخر الملف)

السؤال (6–4): جملة break و else مع حلقة for: محاولات كلمة المرور

اكتب برنامجًا يمنح المستخدم 3 محاولات لإدخال كلمة المرور الصحيحة (py2026).
إذا أدخلها يطبع رقم المحاولة ويتوقّف، وإذا انتهت المحاولات الثلاث دون نجاح يطبع أن الحساب أُغلق.

النتاج المرتبط

(3–1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python).

(3–5) أقارن ناتج تنفيذ جمل التحكم (break، و continue).

(2–2) أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.

✓ الحل

```
PASSWORD = "py2026"
for attempt in range(1, 4):
    guess = input("Enter password: ")
    if guess == PASSWORD:
        print("Access granted on attempt", attempt)
        break
    print("Wrong password")
else:
    print("Account locked after 3 failed attempts")
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 1234، py2026):

```
Enter password: 1234
Wrong password
Enter password: py2026
Access granted on attempt 2
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: python، 2026، abc):

```
Enter password: abc
Wrong password
Enter password: 2026
Wrong password
Enter password: python
Wrong password
Account locked after 3 failed attempts
```

ملاحظات مهمة:

◄ `break` أوقفت الحلقة فور إدخال كلمة المرور الصحيحة، فلم تُطلب المحاولة الثالثة.

◄ جملة `else` المرتبطة بالحلقة تُنفَّذ **فقط** إذا انتهت الحلقة دون تنفيذ `break` (التجربة الثانية).

◄ لاحظ أن `else` هنا تابعة لـ `for` وليست لـ `if`، لذلك كُتبت على مستوى `for` نفسه.

★ تحدٍّ إضافي: ماذا يطبع البرنامج إذا أدخل المستخدم كلمة المرور الصحيحة من المحاولة الأولى؟ وكم مرة نُفّذت الحلقة؟ وهل تُنفَّذ جملة else؟

(إجابة التحدي في الملحق آخر الملف)

السؤال (7-4): المقارنة بين break و continue: المناداة على الطلاب

ينادي المعلّم على أرقام الطلاب من 1 إلى 8، والطالب رقم 5 غائب. نفّذ البرنامجين الآتيين، وقارن بين ناتجيهما.

النتاج المرتبط

(3-5) أقارن ناتج تنفيذ جمل التحكم (break، و continue).

(3-1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python).

✓ الحل

البرنامج (أ) باستخدام continue:

```
for number in range(1, 9):
    if number == 5:
        continue        # تخطَّ هذا الرقم وأكمل الحلقة
    print("Student", number)
```

▶ ناتج التنفيذ:

```
Student 1
Student 2
Student 3
Student 4
Student 6
Student 7
Student 8
```

البرنامج (ب) باستخدام break:

```
for number in range(1, 9):
    if number == 5:
        break           # أوقف الحلقة كاملة
    print("Student", number)
```

▶ ناتج التنفيذ:

```
Student 1
Student 2
Student 3
Student 4
```

وجه المقارنة	continue	break
ماذا تفعل؟	تتخطّى بقية أوامر الدورة الحالية، وتنتقل إلى الدورة التالية	تُوقف الحلقة نهائيًّا، وتنتقل إلى ما بعدها
ناتج المثال	طُبع الطلاب 1 إلى 8 ما عدا 5	طُبع الطلاب 1 إلى 4 فقط
أثرها في else الحلقة	لا تمنع تنفيذها	تمنع تنفيذها

★ **تحدٍّ إضافي:** اكتب برنامجًا يطبع الأعداد من 1 إلى 20 ما عدا مضاعفات العدد 3، ويتوقّف عند أول عدد يقبل القسمة على 2 و 7 معًا.

(إجابة التحدي في الملحق آخر الملف)

السؤال (8-4): جملة else مع حلقة while: لعبة تخمين الرقم

صمّم لعبة تخمين: الرقم السري 7، وللّاعب 3 محاولات. بعد كل تخمين خاطئ يُخبَر هل الرقم السري أكبر أم أصغر. إذا نفدت المحاولات تُطبع رسالة انتهاء اللعبة باستخدام `else` مع حلقة `while`.

النتاج المرتبط

(3-1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python).

(3-2) أستخدم أكثر الهياكل البرمجية (مثل: الحلقات، والجمل الشرطية) مناسبةً لحلّ مشكلات مُعيّنة بكفاءة.

(3-5) أقارن ناتج تنفيذ جمل التحكم (break، و continue). (2-2) أستخدم لغة البرمجة بايثون (Python) في إنشاء برامج تتضمّن جملًا شرطية، وأتتبّع ناتجها وأُنفّذها.	

✓ الحل

```
SECRET = 7
attempts = 0
while attempts < 3:
    guess = int(input("Guess the number (1-10): "))
    attempts += 1
    if guess == SECRET:
        print("Correct! You won in", attempts, "attempts")
        break
    elif guess < SECRET:
        print("Too low")
    else:
        print("Too high")
else:
    print("Game over! The number was", SECRET)
```

▶ ناتج التنفيذ – التجربة 1 (المدخلات: 7 ،9 ،3):

```
Guess the number (1-10): 3
Too low
Guess the number (1-10): 9
Too high
Guess the number (1-10): 7
Correct! You won in 3 attempts
```

▶ ناتج التنفيذ – التجربة 2 (المدخلات: 3 ،2 ،1):

```
Guess the number (1-10): 1
Too low
Guess the number (1-10): 2
Too low
Guess the number (1-10): 3
Too low
Game over! The number was 7
```

ملاحظات مهمة:

◄ هذه اللعبة تطبيق لمُنتَج التعلّم في الوحدة (لعبة تخمين الأرقام)، وتجمع بين الحلقة والجمل الشرطية وجمل التحكم.

★ **تحدٍّ إضافي:** أضف عدّادًا للنقاط: يبدأ اللاعب بـ 30 نقطة، ويخسر 10 نقاط مع كل تخمين خاطئ، ثم اطبع نقاطه عند الفوز.

(إجابة التحدي في الملحق آخر الملف)

السؤال (9-4): الحلقة اللانهائية وجملة pass

أ) لماذا يدخل البرنامج الآتي في حلقة لانهائية؟ صحّحه ليطبع الأعداد من 1 إلى 5.

```
i = 1
while i <= 5:
    print(i)
```

ب) ما ناتج تنفيذ البرنامج الآتي؟ وما فائدة `pass`؟

ج) أحيانًا نكتب `while True` عمدًا ونُنهي الحلقة بـ `break`. اكتب برنامجًا يكرّر طلب أمر من المستخدم ويطبعه، حتى يكتب كلمة exit.

```
for x in range(3):
    pass
print("Done")
```

النتاج المرتبط

(3-1) أكتب جمل التحكم باستخدام الحلقات (مثل: for، و while) في برمجة بايثون (Python).

(3-4) أتعرّف جمل التكرار وأذكر أنواعها.

✓ الحل

أ) البرنامج بعد التصحيح (إضافة تحديث للعدّاد):

```
i = 1
while i <= 5:
    print(i)
    i += 1
```

▶ ناتج التنفيذ:

```
1
2
3
4
5
```

ب) ناتج البرنامج الذي يحتوي على pass:

```
for x in range(3):
    pass
print("Done")
```

▶ ناتج التنفيذ:

```
Done
```

ج) حلقة while True مع break:

```
while True:
    command = input("Type a command (exit to quit): ")
    if command == "exit":
        print("Goodbye")
        break
    print("You typed:", command)
```

▶ ناتج التنفيذ:

```
Type a command (exit to quit): hello
You typed: hello
Type a command (exit to quit): python
You typed: python
Type a command (exit to quit): exit
Goodbye
```

ملاحظات مهمة:

◄ في البرنامج الأصلي لا تتغيّر قيمة `i`، فيبقى الشرط `i <= 5` صحيحًا إلى ما لا نهاية.

◄ `pass` تسمح بكتابة حلقة بلا أوامر (مؤقّتًا)، فتدور الحلقة 3 مرات دون أن تفعل شيئًا، ثم تُطبع "Done".

◄ `while True` حلقة لانهائية **مقصودة**، ولا بدّ أن تحتوي على `break` تُنهيها عند شرط معيّن.

★ تحدٍّ إضافي: أيّ المقاطع الآتية حلقة لانهائية؟ ولماذا؟

```
# المقطع (1)
n = 10
while n > 0:
    n -= 2
# المقطع (2)
n = 1
while n != 10:
    n += 2
# المقطع (3)
n = 1
while n < 100:
    n *= 3
```

(إجابة التحدي في الملحق آخر الملف)

السؤال (10-4): برنامج متكامل: تقرير علامات الشعبة

اكتب برنامجًا متكاملًا **سهل القراءة** يطلب عدد طلاب الشعبة، ثم يطلب علامة كل طالب (من 100) مع التحقّق من صحتها (يُعاد الطلب إذا كانت خارج المدى 0 إلى 100)، ثم يطبع تقريرًا يتضمّن: عدد الناجحين (50 فأكثر)، وعدد الراسبين، ومعدّل الشعبة، وأعلى علامة. ابدأ بكتابة النموذج الأوّلي.

النتاج المرتبط

(3-2) أستخدم أكثر الهياكل البرمجية (مثل: الحلقات، والجمل الشرطية) مناسبةً لحلّ مشكلات مُعيّنة بكفاءة.

	(3-3) أكتب برامج متكاملة على نحوٍ يسهل على الآخرين قراءتها وفهمها. **(1-1)** أُعرّف النموذج الأوّلي للبرنامج.

✓ الحل

النموذج الأوّلي للبرنامج

المدخلات	المعالجة	المخرجات
عدد الطلاب، وعلامة كل طالب	تكرار إدخال العلامات (for)، والتحقّق من صحتها (while)، وجمعها، وعدّ الناجحين والراسبين (if)، وإيجاد أعلى علامة	عدد الناجحين، وعدد الراسبين، ومعدّل الشعبة، وأعلى علامة

```
# ===== تقرير علامات الشعبة =====
PASS_MARK = 50

students = int(input("Number of students: "))
total = 0
passed = 0
failed = 0
highest = 0

for i in range(1, students + 1):
    print("Student", i)
    mark = int(input("Mark: "))
    # التحقّق من أن العلامة بين 0 و 100
    while mark < 0 or mark > 100:
        print("Invalid mark, try again")
        mark = int(input("Mark: "))
    total += mark
    if mark >= PASS_MARK:
        passed += 1
    else:
        failed += 1
    if mark > highest:
        highest = mark

# طباعة التقرير
print("----- Report -----")
print("Passed:", passed)
print("Failed:", failed)
print("Class average:", total / students)
print("Highest mark:", highest)
```

▶ ناتج التنفيذ:

```
Number of students: 4
Student 1
Mark: 75
Student 2
Mark: 120
Invalid mark, try again
Mark: 95
Student 3
Mark: 40
Student 4
Mark: 66
----- Report -----
Passed: 3
Failed: 1
Class average: 69.0
Highest mark: 95
```

ملاحظات مهمة:

◄ **ممارسات تجعل البرنامج سهل القراءة:** أسماء متغيّرات واضحة المعنى (`passed` و `highest`)، وثابت باسم واضح (`PASS_MARK`)، وتعليقات تشرح الأجزاء، وأسطر فارغة تفصل بين أقسام البرنامج.

◄ اخترنا `for` لتكرار الإدخال لأن عدد الطلاب **معروف**، و `while` للتحقّق من صحة العلامة لأن عدد مرات الخطأ **غير معروف**، وهذا اختيار للهيكل البرمجي الأنسب لكل مهمة.

★ **تحدٍّ إضافي:** أضف إلى التقرير: **أدنى** علامة، و**نسبة النجاح المئوية** (عدد الناجحين ÷ عدد الطلاب × 100).

(إجابة التحدي في الملحق آخر الملف)